GA7-220501096-AA4-EV02 Verificación de procedimientos para la definición de componentes front-end de la aplicación (Listas de chequeo)

Ficha:3070324

Programa: análisis y desarrollo de software

juan Santiago Arango puentes

Servicio nacional de aprendizaje Sena

Instructor: FRANCISCO ARNALDO VARGAS

2025

EVIDENCIA DE DESEMPEÑO

GA7-220501096-AA4-EV02

Verificación de procedimientos para la definición de componentes frontend de la aplicación (listas de chequeo)

PORTADA

INSTITUCIÓN EDUCATIVA: SENA - Servicio Nacional de Aprendizaje

PROGRAMA: Técnico en Sistemas

COMPETENCIA: GA7-220501096-AA4-EV02

UNIDAD DE APRENDIZAJE: Desarrollo de aplicaciones frontend con React y TypeScript

TEMA: Definición de componentes frontend de la aplicación Menu Pizza

PRODUCTO: Documento técnico en formato PDF

FECHA: Abril 2026

AUTOR: Equipo de Desarrollo

ÍNDICE

1. Introducción
2. Objetivo general
3. Objetivos específicos
4. Metodología y alcance
5. Contexto del proyecto Menu Pizza
6. Descripción de componentes frontend 6.1 Componente raíz `App` 6.2 Componente `Home` 6.3 Componente `Menu` 6.4 Componente `ShoppingCart` 6.5 Componente `PaymentMethods` 6.6 Componente `Login` 6.7 Componente `Register` 6.8 Componente `LocationForm` 6.9 Componentes administrativos 6.10 Biblioteca de componentes UI reutilizables
7. Justificación de la selección de componentes
8. Listas de chequeo para definición de componentes frontend
9. Conclusiones
10. Referencias

---

1. INTRODUCCIÓN

El presente documento presenta la evidencia de desempeño de la actividad GA7-220501096-AA4-EV02, centrada en la verificación de procedimientos para la definición de componentes frontend en el proyecto Menu Pizza.

Se describe de forma detallada cada componente de React utilizado en la aplicación, su propósito funcional, su rol dentro de la experiencia de usuario y su justificación técnica. El propósito de esta evidencia es demostrar el conocimiento y la trazabilidad en la selección y definición de la arquitectura de componentes frontend.

El documento también incluye listas de chequeo que permiten verificar los procedimientos aplicados durante la definición de componentes, asegurando consistencia con las buenas prácticas del desarrollo frontend moderno y la

implementación de React con TypeScript.

---

2. OBJETIVO GENERAL

Definir y documentar los componentes frontend del proyecto Menu Pizza, presentando una justificación técnica para cada uno de los elementos seleccionados y entregando listas de chequeo que verifiquen los procedimientos seguidos en su definición.

---

3. OBJETIVOS ESPECÍFICOS

- Identificar los componentes React que forman la interfaz de usuario de Menu Pizza.
- Describir la funcionalidad y el propósito de cada componente.
- Justificar la elección de componentes en función de la usabilidad y la arquitectura de la aplicación.
- Documentar procedimientos de verificación mediante listas de chequeo.
- Proveer un documento formal y extenso que sirva como evidencia técnica de la definición de componentes frontend.

---

4. METODOLOGÍA Y ALCANCE

4.1 Metodología

La elaboración de esta evidencia se base en los siguientes pasos:

1. Revisión de la estructura del proyecto Menu Pizza.
2. Identificación de los componentes React presentes en el frontend.
3. Análisis del propósito funcional de cada componente.
4. Clasificación de componentes según su responsabilidad.
5. Redacción de la justificación técnica de cada selección.
6. Construcción de listas de chequeo para verificar procedimientos.

4.2 Alcance

El documento cubre exclusivamente los componentes frontend del proyecto Menu Pizza, incluyendo:

- Componente raíz y de navegación.
- Componentes de presentación de productos.
- Componentes de interacción del usuario.
- Componentes administrativos.
- Componentes de la biblioteca de interfaz reutilizable.

No se abarca la lógica del backend ni la modificación del código existente.

---

5. CONTEXTO DEL PROYECTO MENU PIZZA

Menu Pizza es una aplicación frontend desarrollada con React y TypeScript que permite gestionar pedidos de pizza mediante una experiencia de usuario moderna.

La aplicación incluye:

- Pantalla de inicio con navegación.
- Lista de productos del menú de pizzas.
- Carrito de compras.

- Métodos de pago.
- Autenticación de usuario.
- Registro de nuevos usuarios.
- Registro de ubicación de entrega.
- Panel administrativo con gestión de usuarios, historial, editor de páginas y calendario.

El proyecto utiliza un patrón de componentes basados en React funcional, apoyado en Hooks para la gestión de estado y efectos secundarios. Además, cuenta con una biblioteca de componentes UI reutilizables bajo el directorio `components/ui`.

---

6. DESCRIPCIÓN DE COMPONENTES FRONTEND

La sección siguiente describe con claridad los componentes frontend que intervienen en la aplicación Menu Pizza. Cada componente se presenta con su nombre, responsabilidad, funcionalidad, ejemplo de programación y justificación técnica.

6.1 Componente raíz `App`

Nombre del componente

`App`

Responsabilidad

Actuar como el contenedor principal de la aplicación, gestionar el estado compartido y controlar la navegación entre las diferentes vistas.

Funcionalidad

- Controla qué pantalla se muestra en cada momento.
- Administra el estado del carrito de compras.
- Administra el estado del usuario y del administrador.
- Sincroniza datos con el almacenamiento local.
- Proporciona funciones para agregar, actualizar y eliminar productos en el carrito.

Programación de ejemplo

```
import { useState, useEffect } from 'react';
import type { Product, CartItem, View, User } from './types';
import { Home } from './components/Home';
import { Menu } from './components/Menu';
import { ShoppingCart } from './components/ShoppingCart';

export function App() {
  const [view, setView] = useState<View>('home');
  const [cart, setCart] = useState<CartItem[]>([]);
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    const storedCart = localStorage.getItem('menuPizzaCart');
    if (storedCart) {
      setCart(JSON.parse(storedCart));
    }
  }, []);
```

```
  useEffect(() => {
    localStorage.setItem('menuPizzaCart', JSON.stringify(cart));
  }, [cart]);

  const addToCart = (product: Product) => {
    setCart(prevCart => [...prevCart, { ...product, quantity: 1 }]);
  };

  const navigate = (nextView: View) => {
    setView(nextView);
  };

  return (
    <div className="app-container">
      {view === 'home' && <Home user={user} onNavigate={navigate} />}
      {view === 'menu' && <Menu onAddToCart={addToCart} cartItemsCount={cart.length} onBack=
{() => navigate('home')} />}
      {view === 'cart' && <ShoppingCart cart={cart} onBack={() => navigate('menu')}
onUpdateQuantity={() => {}} onRemoveItem={() => {}} />}
    </div>
  );
}
```

Justificación técnica

`App` debe ser el componente raíz porque centraliza el estado compartido y la navegación. Esta estructura facilita la coordinación entre vistas y el uso de Hooks sin generar lógica dispersa.

6.2 Componente `Home`

Nombre del componente

`Home`

Responsabilidad

Presentar la pantalla de bienvenida e iniciar la interacción del usuario con el sistema.

Funcionalidad

- Muestra un saludo al usuario.
- Presenta botones de acceso a otras secciones.
- Indica el estado de la sesión del usuario.
- Guiar al usuario hacia el proceso de compra.

Programación de ejemplo

```
import type { User, View } from '../types';

interface HomeProps {
  user: User | null;
  onNavigate: (nextView: View) => void;
}
```

```
export function Home({ user, onNavigate }: HomeProps) {
  return (
    <main className="home-page">
      <h1>Bienvenido a Menu Pizza</h1>
      <p>Ordena tus pizzas favoritas de forma rápida y segura.</p>
      <div className="home-actions">
        <button onClick={() => onNavigate('menu')}>Ver menú</button>
        <button onClick={() => onNavigate('cart')}>Ver carrito</button>
        {!user ? (
          <button onClick={() => onNavigate('login')}>Iniciar sesión</button>
        ) : (
          <p>Usuario: {user.name}</p>
        )}
      </div>
    </main>
  );
}
```

Justificación técnica

Este componente es la primera experiencia del usuario, por lo que debe ser simple, clara y facilitar el acceso inmediato a las funciones principales.

6.3 Componente `Menu`

Nombre del componente

`Menu`

Responsabilidad

Mostrar el catálogo de productos y permitir su selección y adición al carrito.

Funcionalidad

- Lista productos con imagen, nombre, descripción y precio.
- Permite agregar productos al carrito.
- Indica cuántos productos contiene el carrito.
- Ofrece una acción de retorno a la pantalla anterior.

Programación de ejemplo

```
import type { Product } from '../types';

interface MenuProps {
  onAddToCart: (product: Product) => void;
  cartItemsCount: number;
  onBack: () => void;
}

const products: Product[] = [
  { id: '1', name: 'Pizza Hawaiana', description: 'Jamón y piña', price: 46000, image:
'/images/hawaiana.jpg' },
```

```
  { id: '2', name: 'Pizza Peperoni', description: 'Pepperoni dorado', price: 42000, image:
'/images/peperoni.jpg' },
];

export function Menu({ onAddToCart, cartItemsCount, onBack }: MenuProps) {
  return (
    <section className="menu-page">
      <header>
        <button onClick={onBack}>Volver</button>
        <span>Carrito: {cartItemsCount}</span>
      </header>
      <div className="product-list">
        {products.map(product => (
          <article key={product.id} className="product-card">
            <img src={product.image} alt={product.name} />
            <h2>{product.name}</h2>
            <p>{product.description}</p>
            <p>{product.price}</p>
            <button onClick={() => onAddToCart(product)}>Agregar al carrito</button>
          </article>
        ))}
      </div>
    </section>
  );
}
```

Justificación técnica

`Menu` agrupa la presentación de productos y la interacción de selección en un único componente, lo que mejora la claridad y el mantenimiento.

6.4 Componente `ShoppingCart`

Nombre del componente

`ShoppingCart`

Responsabilidad

Administrar la vista del carrito y las operaciones de edición de cantidades y eliminación.

Funcionalidad

- Muestra productos agregados con subtotal.
- Ajusta cantidades de cada producto.
- Elimina productos del carrito.
- Indica el total acumulado de la orden.

Programación de ejemplo

```
import type { CartItem } from '../types';

interface ShoppingCartProps {
  cart: CartItem[];
```

```
  onBack: () => void;
  onUpdateQuantity: (id: string, quantity: number) => void;
  onRemoveItem: (id: string) => void;
}

export function ShoppingCart({ cart, onBack, onUpdateQuantity, onRemoveItem }:
ShoppingCartProps) {
  const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);

  return (
    <section className="shopping-cart-page">
      <button onClick={onBack}>Volver al menú</button>
      <h1>Carrito de Compras</h1>
      {cart.length === 0 ? (
        <p>No hay productos en el carrito.</p>
      ) : (
        <>
          {cart.map(item => (
            <div key={item.id} className="cart-item">
              <h2>{item.name}</h2>
              <p>Cantidad: {item.quantity}</p>
              <p>Precio unitario: {item.price}</p>
              <div className="cart-actions">
                <button onClick={() => onUpdateQuantity(item.id, item.quantity + 1)}>+
</button>
                <button onClick={() => onUpdateQuantity(item.id, item.quantity - 1)}>-
</button>
                <button onClick={() => onRemoveItem(item.id)}>Eliminar</button>
              </div>
            </div>
          ))}
          <p>Total: {total}</p>
        </>
      )}
    </section>
  );
}
```

Justificación técnica

Separar el carrito en su propio componente permite una interfaz clara para la gestión de pedidos, sin mezclar lógica de selección de productos con acciones de compra.

6.5 Componente `PaymentMethods`

Nombre del componente

`PaymentMethods`

Responsabilidad

Presentar las opciones de pago y controlar el paso final de confirmación de la orden.

Funcionalidad

- Muestra el monto total.
- Ofrece alternativas de pago.
- Activa la confirmación cuando el usuario elige una opción.

Programación de ejemplo

```
interface PaymentMethodsProps {
  total: number;
  onConfirm: () => void;
}

export function PaymentMethods({ total, onConfirm }: PaymentMethodsProps) {
  return (
    <section className="payment-page">
      <h1>Métodos de pago</h1>
      <p>Total a pagar: {total}</p>
      <div className="payment-options">
        <button onClick={onConfirm}>Pagar con tarjeta</button>
        <button onClick={onConfirm}>Pagar contra entrega</button>
      </div>
    </section>
  );
}
```

Justificación técnica

El pago es una etapa crítica, por eso se debe aislar en un componente propio que permita controlar la lógica de confirmación sin afectar el resto del flujo.

6.6 Componente `Login`

Nombre del componente

`Login`

Responsabilidad

Validar el acceso de usuarios existentes y capturar credenciales.

Funcionalidad

- Captura correo y contraseña.
- Envía los datos para validación.
- Muestra mensajes de error si la autenticación falla.

Programación de ejemplo

```
import { useState } from 'react';

interface LoginProps {
  onLogin: (email: string, password: string) => void;
}
```

```
export function Login({ onLogin }: LoginProps) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (event: React.FormEvent) => {
    event.preventDefault();
    onLogin(email, password);
  };

  return (
    <form className="login-form" onSubmit={handleSubmit}>
      <h1>Iniciar Sesión</h1>
      <input type="email" value={email} onChange={e => setEmail(e.target.value)}
placeholder="Correo" required />
      <input type="password" value={password} onChange={e => setPassword(e.target.value)}
placeholder="Contraseña" required />
      <button type="submit">Ingresar</button>
    </form>
  );
}
```

Justificación técnica

Separar el login como componente independiente permite concentrar la seguridad y validación en un único lugar.

6.7 Componente `Register`

Nombre del componente

`Register`

Responsabilidad

Capturar los datos de nuevos usuarios y crear cuentas.

Funcionalidad

- Recibe nombre, correo y contraseña.
- Valida la información ingresada.
- Envía el registro al proceso correspondiente.

Programación de ejemplo

```
import { useState } from 'react';

interface RegisterProps {
  onRegister: (name: string, email: string, password: string) => void;
}

export function Register({ onRegister }: RegisterProps) {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
```

```
  const handleSubmit = (event: React.FormEvent) => {
    event.preventDefault();
    onRegister(name, email, password);
  };

  return (
    <form className="register-form" onSubmit={handleSubmit}>
      <h1>Registro</h1>
      <input value={name} onChange={e => setName(e.target.value)} placeholder="Nombre"
required />
      <input type="email" value={email} onChange={e => setEmail(e.target.value)}
placeholder="Correo" required />
      <input type="password" value={password} onChange={e => setPassword(e.target.value)}
placeholder="Contraseña" required />
      <button type="submit">Registrar</button>
    </form>
  );
}
```

Justificación técnica

El registro debe ser un flujo propio para evitar mezclarlo con el inicio de sesión y con la gestión del carrito.

6.8 Componente `LocationForm`

Nombre del componente

`LocationForm`

Responsabilidad

Capturar la dirección de entrega y datos adicionales para el pedido.

Funcionalidad

- Recibe dirección, ciudad y observaciones.
- Valida los campos obligatorios.
- Envía la información al almacenamiento correspondiente.

Programación de ejemplo

```
import { useState } from 'react';

interface LocationFormProps {
  onSaveLocation: (address: string, city: string, notes: string) => void;
}

export function LocationForm({ onSaveLocation }: LocationFormProps) {
  const [address, setAddress] = useState('');
  const [city, setCity] = useState('');
  const [notes, setNotes] = useState('');

  const handleSubmit = (event: React.FormEvent) => {
    event.preventDefault();
```

```
    onSaveLocation(address, city, notes);
  };

  return (
    <form className="location-form" onSubmit={handleSubmit}>
      <h1>Dirección de entrega</h1>
      <input value={address} onChange={e => setAddress(e.target.value)}
placeholder="Dirección" required />
      <input value={city} onChange={e => setCity(e.target.value)} placeholder="Ciudad"
required />
      <textarea value={notes} onChange={e => setNotes(e.target.value)}
placeholder="Información adicional" />
      <button type="submit">Guardar ubicación</button>
    </form>
  );
}
```

Justificación técnica

Este componente aísla la captura de datos de entrega para mantener la lógica separada de los formularios de usuario y pago.

6.9 Componentes administrativos

6.9.1 `AdminPanel`

Responsabilidad: Proveer un punto de acceso a las funcionalidades administrativas.

Programación de ejemplo

```
import type { View } from '../types';

interface AdminPanelProps {
  onNavigate: (view: View) => void;
}

export function AdminPanel({ onNavigate }: AdminPanelProps) {
  return (
    <section className="admin-panel">
      <h1>Panel Administrativo</h1>
      <button onClick={() => onNavigate('adminUsers')}>Gestión de usuarios</button>
      <button onClick={() => onNavigate('adminEditor')}>Editor de páginas</button>
      <button onClick={() => onNavigate('adminHistory')}>Historial</button>
      <button onClick={() => onNavigate('adminCalendar')}>Calendario</button>
    </section>
  );
}
```

Justificación: Centraliza la administración de la aplicación y actúa como menú para las vistas administrativas.

6.9.2 `UserManagement`

Responsabilidad: Gestionar usuarios registrados, estados y privilegios.

Programación de ejemplo

```
export function UserManagement() {
  return (
    <section className="user-management">
      <h1>Gestión de Usuarios</h1>
      <p>Aquí se pueden ver y editar los datos de los usuarios.</p>
    </section>
  );
}
```

Justificación: Permite la administración de cuentas dentro de un módulo aislado.

6.9.3 `PageEditor`

Responsabilidad: Editar el contenido de las páginas internas.

Programación de ejemplo

```
export function PageEditor() {
  return (
    <section className="page-editor">
      <h1>Editor de Página</h1>
      <textarea placeholder="Escribe el contenido de la página aquí..."></textarea>
      <button>Guardar cambios</button>
    </section>
  );
}
```

Justificación: Separa la modificación de contenido de la lógica de usuario y pedido.

6.9.4 `UserHistory`

Responsabilidad: Mostrar el historial de pedidos de un usuario.

Programación de ejemplo

```
export function UserHistory() {
  return (
    <section className="user-history">
      <h1>Historial de Usuario</h1>
      <p>Listado de pedidos anteriores del usuario.</p>
    </section>
  );
}
```

Justificación: Facilita la consulta de transacciones pasadas en una vista específica.

6.9.5 `AdminHistory`

Responsabilidad: Registrar eventos importantes de administración.

Programación de ejemplo

```
export function AdminHistory() {
  return (
    <section className="admin-history">
      <h1>Historial Administrativo</h1>
      <p>Registro de acciones realizadas por el administrador.</p>
    </section>
  );
}
```

Justificación: Soporta la auditoría de acciones administrativas.

6.9.6 `CalendarView`

Responsabilidad: Mostrar datos cronológicos o fechas relevantes del sistema.

Programación de ejemplo

```
export function CalendarView() {
  return (
    <section className="calendar-view">
      <h1>Calendario</h1>
      <p>Visualización de fechas y eventos importantes.</p>
    </section>
  );
}
```

Justificación: Permite la visualización de agenda y fechas dentro del panel administrativo.

6.10 Biblioteca de componentes UI reutilizables

Nombre del directorio

`components/ui`

Responsabilidad

Contiene componentes de interfaz reutilizables como botones, formularios, tarjetas, menús, modales, tablas, controles y elementos de navegación.

Funcionalidad

- Proveer una base consistente de estilos y comportamiento.
- Reducir duplicación de código.
- Mejorar la consistencia visual de la aplicación.
- Facilitar la construcción de nuevas páginas con elementos reutilizados.

Programación de ejemplo

```
interface ButtonProps {
  variant?: 'primary' | 'secondary' | 'danger';
  size?: 'sm' | 'md' | 'lg';
  children: React.ReactNode;
  onClick?: () => void;
}
```

```
export function Button({
  variant = 'primary',
  size = 'md',
  children,
  onClick
}: ButtonProps) {
  const baseClasses = 'rounded px-4 py-2 font-medium';
  const variantClasses = {
    primary: 'bg-orange-600 text-white',
    secondary: 'bg-gray-200 text-gray-900',
    danger: 'bg-red-600 text-white'
  };
  const sizeClasses = {
    sm: 'text-sm',
    md: 'text-base',
    lg: 'text-lg'
  };

  return (
    <button className={`${baseClasses} ${variantClasses[variant]} ${sizeClasses[size]}`}
onClick={onClick}>
      {children}
    </button>
  );
}
```

Justificación técnica

Una biblioteca de componentes UI es fundamental para mantener la coherencia en el diseño y acelerar el desarrollo. Permite aplicar principios de diseño atómico y componentes componibles.

---

7. JUSTIFICACIÓN DE LA SELECCIÓN DE COMPONENTES

7.1 Principios aplicados

La selección de componentes se hizo con base en los siguientes principios:

- **Responsabilidad única:** Cada componente debe tener una sola responsabilidad bien definida.
- **Reutilización:** Componentes UI comunes se deben reutilizar en varias partes de la aplicación.
- **Escalabilidad:** Los componentes deben ser fáciles de extender para nuevas funcionalidades.
- **Separación de lógica y presentación:** Componentes de presentación no deben contener lógica de negocio compleja.
- **Facilidad de mantenimiento:** Arquitectura modular para cambios y correcciones futuras.
- **Claridad en la navegación:** Componentes de vista distintivos permiten un flujo de usuario claro.

7.2 Uso de componentes funcionales con Hooks

La aplicación utiliza componentes funcionales porque:

- Permiten el uso directo de Hooks (`useState` , `useEffect`).
- Son más simples de testear y depurar.
- Reducen la complejidad asociada a componentes de clase.

- Mejoran la compatibilidad con las versiones modernas de React.

7.3 Criterios de diseño para Menu Pizza

Para seleccionar los componentes se consideraron los siguientes criterios específicos del proyecto:

- El frontend debe ser intuitivo para la selección de productos.
- El flujo de compra debe ser claro y secuencial.
- El manejo del carrito debe ser visible y editable.
- La autenticación y el registro deben ser modulares.
- La administración debe estar separada del flujo del cliente.
- La interfaz debe poder adaptarse a cambios de diseño y contenido.

---

8. LISTAS DE CHEQUEO PARA DEFINICIÓN DE COMPONENTES FRONTEND

8.1 Checklist de definición de componente

- ☑ El componente tiene un nombre descriptivo en PascalCase.
- ☑ El componente cumple una única responsabilidad.
- ☑ El componente es reutilizable cuando aplica.
- ☑ El componente recibe props tipadas con TypeScript.
- ☑ El componente realiza la menor cantidad de lógica posible en la vista.
- ☑ El componente usa Hooks de manera correcta.
- ☑ El componente no modifica el estado global sin un método claro.
- ☑ El componente se integra a la navegación de la aplicación.
- ☑ El componente tiene justificación técnica documentada.
- ☑ El componente responde al requerimiento de la experiencia de usuario.

8.2 Checklist de verificación de procedimientos

Procedimiento	Verificación	Resultado
Identificar componentes principales	Se listaron todos los componentes de la ruta `components/`	✅
Separar vistas de lógica	Las vistas y la lógica de estado se mantienen en `App` y componentes específicos	✅
Definir componentes de presentación	`Home`, `Menu`, `ShoppingCart`, `PaymentMethods`	✅
Definir componentes de formulario	`Login`, `Register`, `LocationForm`	✅
Definir componentes administrativos	`AdminPanel`, `UserManagement`, `PageEditor`, `UserHistory`, `AdminHistory`, `CalendarView`	✅
Identificar biblioteca UI reusable	Directorio `components/ui`	✅
Aplicar justificación técnica	Cada componente incluye una justificación en el documento	✅

Uso de TypeScript para tipado	Se utilizan interfaces y tipos en los componentes	✅
Asegurar flujo de navegación	Se describen las vistas y transiciones entre ellas	✅
Generar evidencia formal	Documento con portada, índice, introducción, objetivos y conclusiones	✅

8.3 Checklist del componente `App`

- ☑ Controla vistas y rutas internas.
- ☑ Gestiona estado global compartido.
- ☑ Sincroniza el estado con localStorage.
- ☑ Define funciones para agregar, eliminar y actualizar productos.
- ☑ Mantiene la lógica de navegación fuera de componentes de presentación.

8.4 Checklist del componente `Menu`

- ☑ Presenta lista de productos.
- ☑ Permite agregar productos al carrito.
- ☑ Muestra conteo de elementos del carrito.
- ☑ Separa la visualización de producto de la lógica de carrito.
- ☑ Es compatible con el estado global del carrito.

8.5 Checklist del componente `ShoppingCart`

- ☑ Muestra productos agregados y cantidades.
- ☑ Permite modificar cantidades.
- ☑ Permite eliminar productos.
- ☑ Muestra resumen de pago.
- ☑ Prepara la transición al componente de pago.

8.6 Checklist de componentes administrativos

- ☑ Cada vista administrativa tiene un componente dedicado.
- ☑ Los componentes administrativos están separados de la lógica de usuario normal.
- ☑ El panel administrativo actúa como concentrador.
- ☑ La información sensible se gestiona en módulos específicos.

---

9. CONCLUSIONES

El análisis de los componentes frontend del proyecto Menu Pizza demuestra una arquitectura modular y coherente con los principios de desarrollo moderno en React y TypeScript.

Los componentes han sido seleccionados con base en su responsabilidad única, su capacidad de reutilización y su adecuación a los procesos de compra en línea. El componente raíz `App` centraliza el estado y la navegación, mientras que las vistas específicas como `Menu`, `ShoppingCart` y `PaymentMethods` preservan claridad en el ciclo de pedido.

La existencia de un conjunto de componentes administrativos garantiza que la administración y la experiencia del cliente se mantengan separadas, lo cual es esencial para escalabilidad y seguridad.

Las listas de chequeo incluidas en este documento verifican que los procedimientos se aplicaron correctamente y que cada componente está bien definido según las características del software a desarrollar.

---

10. REFERENCIAS

- Documentación oficial de React: https://react.dev
- Documentación oficial de TypeScript: https://www.typescriptlang.org
- Patrones de diseño de componentes React
- Buenas prácticas de organización de carpetas en proyectos React
- Principios SOLID aplicados a frontend

Documento generado: Abril 2026

Evidencia: GA7-220501096-AA4-EV02

Formato de entrega: PDF